FinOps Cost Cockpit

Feature Catalogue and Proof-of-Concept Development Plan

Databricks FinOps Accelerator

Document	Feature catalogue and POC development plan
Prepared by	Navya, Nikhil
Version	1.1 (cockpit delivered as a Streamlit application)
Date	October 2026
Status	Draft for mentor review

Contents

1. Purpose and scope
2. Feature catalogue
3. Status summary
4. Roles and responsibilities
5. Development plan
6. Interface contract
7. Application architecture
8. Design rules
9. Ways of working
10. Risks and mitigations
11. Deliverables

1. Purpose and scope

This document defines the features of the FinOps Cost Cockpit proof of concept (POC) and the plan for developing it. The cockpit turns Databricks system tables into an actionable view of cost: where money is spent, who owns the spend, and where compute is wasted.

The feature list was produced by combining three inputs: the seven features in the Cost Cockpit design, the capabilities of the existing open-source Databricks FinOps Accelerator, and a gap analysis between the two. Duplicate features were merged and six overlapping features were resolved through the design rules in Section 8, resulting in twelve unique features. The cockpit itself is delivered as a Streamlit application with a modular feature architecture, described in Section 7.

The POC is planned over four weeks by a team of two, assuming two to three hours of work per person per day. Development and testing use Databricks Free Edition together with synthetic data.

2. Feature catalogue

Table 1 lists the twelve features. Status indicates the starting point for each feature: **Already works** means the capability runs today in the accelerator; **Needs extension** means it exists in part and requires new logic; **Build new** means it is not present in the repository. Source indicates the origin of the feature: the original accelerator (Repo), the Cost Cockpit design (Plan), or components added during testing (Team).

ID	Feature	Description	Source	Status
A. Cost and attribution				
F1	**Cost base**	A daily job joins system.billing.usage with system.billing.list_prices to convert DBUs into dollars, written to cockpit.cost_base.	Repo and Plan	Already works
F2	**Tag attribution and tag quality**	Groups cost by team, project and owner from custom tags (including alias keys) and identifies spend that is missing required tags.	Repo and Plan	Already works
F3	**Shared warehouse catalog allocation**	Allocates shared SQL warehouse cost across business domains according to each catalog's share of query execution time in system.query.history.	Plan	Build new
B. Waste detection				
F4	**All-purpose compute misuse**	A single flag with three reasons: BI tools connecting through JDBC/ODBC (system.access.audit), scheduled jobs running on all-purpose SKUs, and many users sharing one cluster. Reports the dollars lost against cheaper compute.	Plan and Repo	Needs extension
F5	**Idle and zombie compute**	Identifies clusters with auto-termination disabled or above 60 minutes (system.compute.clusters) and clusters with low CPU and memory utilisation. Does not recommend downsizing memory-bound workloads.	Plan and Repo	Needs extension
F6	**Job failures and failed-run cost**	Reports failure rate, retries and the cost of failed job runs from the Lakeflow system tables.	Repo	Already works
C. Prioritisation				
F7	**Unified waste backlog**	A single ranked list of all flags with a 0-100 priority score, confidence level and evidence, without double-counting savings.	Repo	Needs extension
F8	**Spend trend and week-over-week change**	Daily cost trend and the change in spend between the current and previous week for each team.	Repo and Plan	Needs extension
D. Reporting and alerting				
F9	**Streamlit cost cockpit**	A Streamlit application in which every page is a self-contained feature module that can be added, updated or removed without changing shared code. Runs locally on synthetic data and is deployed as a Databricks App.	Team	Build new
F10	**Weekly alert to team leads**	A Databricks SQL Alert that emails team leads every Monday with their change in spend and their top three waste flags.	Plan	Build new
E. Data quality and testing				

ID	Feature	Description	Source	Status
F11	**Data health and coverage checks**	Verifies which system tables are available and how well prices were matched, and continues with limited outputs when optional sources are missing.	Repo	Already works
F12	**Synthetic data sandbox**	Realistic synthetic system tables that allow the full pipeline to be tested on Databricks Free Edition and on a local machine.	Team	Needs extension

Table 1. Feature catalogue

3. Status summary

Table 2 summarises the starting status of the twelve features.

Status	Count	Features
Already works	4	F1, F2, F6, F11
Needs extension	5	F4, F5, F7, F8, F12
Build new	3	F3, F9, F10
Total	**12**	

Table 2. Status summary

Table 3 lists the next step required for each feature.

ID	Next step
F1	Rename outputs to the cockpit schema and enable the daily schedule.
F2	Apply rule R2 for shared SQL warehouses.
F3	Build and validate on real data (486 queries currently available in system.query.history).
F4	Combine the three signals and count savings once per resource (rule R1).
F5	Add the auto-termination rule to the existing utilisation check.
F6	Correct the retry count by joining task runs on job_run_id.
F7	Write all flags to cockpit.waste_flags with priority scores.
F8	Create cockpit.weekly_spend_delta.
F9	Application shell and six initial pages are in place; add pages for F3, F4, F5, F7 and F8 as feature modules and deploy as a Databricks App.
F10	Build the alert and test it by sending it to the team first.
F11	Add checks for system.access.audit, system.compute.clusters and system.query.history.
F12	Add synthetic audit, cluster and query history tables.

Table 3. Next steps by feature

4. Roles and responsibilities

Work is divided by functional area rather than by development stage, so that each team member owns their features end to end, from SQL logic to cockpit page, and the two members do not modify the same files. The only dependency between the two areas is the interface table described in Section 6.

	Navya	**Nikhil**
Area	Cost, attribution and reporting	Waste detection
Question answered	Where is the money going, and who owns it?	Where is money being wasted?
Features	F1 Cost base F2 Tag attribution and tag quality F3 Shared warehouse catalog allocation F8 Spend trend and week-over-week change F9 Cost cockpit F10 Weekly alert to team leads	F4 All-purpose compute misuse F5 Idle and zombie compute F6 Job failures and failed-run cost F7 Unified waste backlog F12 Synthetic data sandbox
Shared	F11 Health checks for her tables	F11 Health checks for his tables
Primary data	Real account data (billing, prices, query history)	Synthetic data (audit, clusters, jobs)
Rationale	Already familiar with the environment, the demonstration and the cockpit application.	The waste flags depend on synthetic data, so owning both avoids waiting on another person.

Table 4. Responsibilities

5. Development plan

Development runs in two parallel tracks over four weeks. Table 5 gives an overview; Tables 6 to 9 give the tasks for each week.

Week	**Theme**	**Exit criterion**
1	Setup and foundations	Both environments run the pipeline; repository on GitHub; synthetic data extended; interface agreed.
2	Core feature development	Each member's new tables are created and correct on synthetic data.
3	Integration	A single run produces all outputs; the cockpit application shows the new pages; test alert is received.
4	Testing, documentation and demonstration	POC complete and demonstrated to the mentor.

Table 5. Plan overview

Week 1: Setup and foundations

Navya	Nikhil
1. Create the GitHub repository from the current project folder, including the sandbox, and grant Nikhil access.	1. Clone the repository, set up the local environment and run the local pipeline.
2. F1: Rename outputs to the cockpit schema (daily_cost to cost_base).	2. F12: Add synthetic system.access.audit, system.compute.clusters and system.query.history tables.
3. Draft the cockpit.waste_flags interface with Nikhil.	3. Review and approve the cockpit.waste_flags interface.
4. Review the columns and catalogs available in system.query.history.	4. Add test scenarios: one BI tool on all-purpose compute, one scheduled job on all-purpose compute, two zombie clusters.
Joint activity: Kick-off meeting (30 minutes) to agree rules R1 to R5; establish a 15-minute daily check-in.	

Table 6. Week 1: Setup and foundations

Week 2: Core feature development

Navya	Nikhil
1. F3: Allocate warehouse cost by percentage of query execution time per catalog, validated first on real data.	1. F4: Build the all-purpose misuse flag (JDBC/ODBC, scheduled jobs, shared users), with dollars lost calculated as DBUs multiplied by the difference between the all-purpose and jobs compute prices.
2. F2: Integrate catalog allocation with tag attribution (rule R2).	2. F5: Build the zombie cluster rule (auto-termination disabled or above 60 minutes) and merge it with the utilisation check.
3. F8: Create the week-over-week spend change per team (cockpit.weekly_spend_delta).	3. F6: Correct the retry count by joining on job_run_id.
Joint activity: Mid-week review session to review each other's SQL.	

Table 7. Week 2: Core feature development

Week 3: Integration

Navya	Nikhil
1. F9: Add the catalog allocation (F3) and weekly change (F8) pages as Streamlit feature modules.	1. F7: Write all flags into cockpit.waste_flags with priority scores (rules R1 and R5).
2. F10: Create the Monday SQL Alert and send it to the team for testing.	2. F9: Add the waste flags page (F4, F5, F7) as a Streamlit feature module.
3. F11: Add health checks for her new tables.	3. F11: Add health checks for the audit, cluster and query history sources.
	4. Write unit tests for every new flag and page, following the existing test structure.
Joint activity: Enable the daily schedule (F1) and complete a full run on synthetic and real data.	

Table 8. Week 3: Integration

Week 4: Testing, documentation and demonstration

Navya	Nikhil
1. F9: Deploy the cockpit as a Databricks App and write the guide for adding, updating and removing pages.	1. Verify that every synthetic scenario is flagged correctly.
2. Update the README with the architecture, features and run instructions.	2. Write docs/waste_flags.md describing each flag, its rule and an example.
3. Prepare the demonstration script.	3. Prepare the demonstration of the defect correction and the savings calculation.
Joint activity: Rehearse the demonstration twice and present it to the mentor.	

Table 9. Week 4: Testing, documentation and demonstration

6. Interface contract

The waste detection features (F4 to F7) feed the cockpit application and the weekly alert (F9 and F10). To allow both tracks to proceed in parallel, the structure of the shared table cockpit.waste_flags is fixed in Week 1. Each row represents one flag.

Column	Description and example
flag_type	Type of flag: ALL_PURPOSE_MISUSE, ZOMBIE_CLUSTER, IDLE_COMPUTE, FAILED_JOBS, MISSING_TAGS.
reason	Specific reason: BI_TOOL_JDBC, SCHEDULED_JOB, SHARED_USERS, AUTO_TERM_OFF.
resource_type, resource_id	The affected resource, for example CLUSTER and 0101-shared-ds-cluster.
team, owner	Responsible team and owner, for example data-science and frank@example.com.
monthly_cost	Estimated monthly cost of the resource, for example 1509.60.
estimated_savings	Estimated monthly saving, counted once per resource (rule R1), for example 980.00.
priority_score	Priority from 0 to 100.
confidence	HIGH, MEDIUM or LOW.
evidence	Plain-language explanation, for example "3 users, 12% CPU, job on all-purpose compute".
run_id, created_at	Identifier and timestamp of the pipeline run, for traceability.

Table 10. Structure of cockpit.waste_flags

7. Application architecture

The cockpit is a Streamlit application rather than a native AI/BI (Lakeview) dashboard. A code-based application keeps the user interface under version control, allows custom interaction, and lets features be added, updated or removed independently. The system is organised in three layers with one-way dependencies.

Layer	Responsibility	Location
Engine	Reads Databricks system tables and writes curated (gold) tables. All business logic lives here, in Python orchestrating SQL templates.	src/databricks_finops (upstream, unchanged) and src/cockpit (new steps)
Gold tables	The contract between engine and application: summary tables, views and cockpit.waste_flags.	Databricks schema; CSV export for local work
Application	Presents the gold tables. It never reads system tables and never runs Spark.	app/

Table 11. Application layers

Each cockpit page is a feature module: a folder under app/cockpit_ui/features that declares its identifier, title, owner, the gold tables it requires and a render function. The application discovers modules automatically. A page whose tables are not yet available shows a notice instead of failing, and an error in one page is contained to that page. The same code reads local CSV files or a Databricks SQL warehouse, selected by a single setting.

Change	How it is made
Add a page	Create a folder under app/cockpit_ui/features that exports FEATURE. No shared code is changed.
Update a page	Edit only that page's folder; the page has a single owner (see CODEOWNERS).
Remove or hide a page	Delete its folder, or set enabled: false for it in app/features.yaml.
Add pipeline logic	Create a module under src/cockpit/steps that exports STEP, with its SQL in src/cockpit/sql. Steps run after the upstream pipeline in dependency order.
Change the data source	Set COCKPIT_SOURCE to local (synthetic CSV) or databricks (SQL warehouse).

Table 12. Working with feature modules

8. Design rules

Six overlaps were identified between the Cost Cockpit design and the existing accelerator. Rules R1 to R5 resolve them; rule R6 governs the application architecture. All rules apply to all development.

Rule	Description	Owner
R1	A resource's savings are counted once, even when it is flagged for several reasons.	Nikhil
R2	For shared SQL warehouses, catalog allocation takes precedence over tag attribution.	Navya
R3	The existing accelerator engine is retained (Python orchestrating SQL templates). New features are added as new SQL files.	Both
R4	All outputs are written to the cockpit schema (for example cockpit.cost_base and cockpit.waste_flags).	Both
R5	The three flags in the weekly alert are the three with the highest priority score.	Navya
R6	Cockpit pages read only gold tables through the application's data-source layer; calculations belong in the engine, not in pages.	Both

Table 13. Design rules

9. Ways of working

Practice	Approach
Source control	GitHub (private repository), with one branch per feature (for example navya/f3-catalog-allocation). Changes reach main only through pull requests.
Code review	Each member reviews the other's pull requests before they are merged; CODEOWNERS routes each feature folder to its owner. Automated tests run on every pull request.
Communication	A 15-minute daily check-in covering progress, next steps and blockers.
Environments	Each member uses their own Databricks Free Edition workspace. The synthetic data uses a fixed seed, so results are identical across workspaces.
Definition of done	A feature is complete when it has its SQL logic, a unit test, a synthetic scenario that triggers it, a health check and a cockpit page.

Table 14. Ways of working

10. Risks and mitigations

Risk	Mitigation
system.query.history does not expose a catalog column.	Derive the catalog from the query text, or use synthetic data.
No JDBC/ODBC connections exist in the real audit log (none found).	Use synthetic data for the demonstration and state this clearly.
SQL Alert email delivery is limited on Free Edition.	Demonstrate the alert query and the alert configuration.
system.compute.clusters is empty on Free Edition.	Use synthetic data; production accounts populate this table.
Databricks Apps is limited or unavailable on Free Edition.	Run the same application locally against the SQL warehouse or the synthetic data; no code change is needed.
One member falls behind schedule.	Use the Week 4 buffer; the F7 to F9 interface can be developed against sample rows.

Table 15. Risks and mitigations

11. Deliverables

At the end of the POC the team will deliver:

1. A GitHub repository containing the code, tests, synthetic data sandbox and documentation.
2. The cockpit schema in Databricks containing all output tables.
3. The Streamlit Cost Cockpit, deployed as a Databricks App, with pages for cost, catalog allocation, waste flags and weekly change.
4. The Monday alert email (test version).
5. A guide for adding, updating and removing cockpit pages and pipeline steps.
6. The corrected retry calculation and the estimated savings figures for the mentor demonstration.